Developing Intelligent Systems with COH and GISMOL

Version: 0.1.0
Date: March 2026
Author: Harris Wang
Contact: harrisw@athabascau.ca
Repository: https://github.com/hongxueharriswang/gismol_toolkit

---

Welcome! This tutorial will guide you through the process of designing, building, simulating, and learning with intelligent systems using the **Constrained Object Hierarchies (COH)** theoretical framework and the **GISMOL** Python toolkit. By the end, you'll be able to create your own hierarchical, constraint-driven models and run experiments on them.

The tutorial assumes you have basic knowledge of Python and object-oriented programming. Familiarity with PyTorch is helpful for the learning sections but not strictly required.

Table of Contents

1. Introduction to COH and GISMOL
2. Installation and First Steps
3. Your First COH System: A Moving Point
4. The COH 9-tuple in GISMOL – A Detailed Walkthrough
    - Components (C)
    - Attributes (A)
    - Methods (M)
    - Neural Components (N)
    - Embedding (E)
    - Identity Constraints (I)
    - Trigger Constraints (T)
    - Goal Constraints (G)
    - Daemons (D)
5. Building a Hierarchy: The Smart Home Example
6. Adding Constraints to Enforce Behavior
7. Learning with Neural Components
8. Monitoring with Daemons
9. Simulation in Depth
    - The Simulator Class
    - Policies and Action Selection
    - Event Handling
    - Real-Time Simulation
10. Advanced Composition: Categorical Operations
    - Product (Parallel Composition)
    - Coproduct and Exponential (Experimental)
11. Saving and Loading Your Models

12. Visualizing the Hierarchy
13. Debugging and Testing Tips
14. Extending GISMOL
15. Complete Case Study: Gridworld Navigator
16. Conclusion and Next Steps

---

1. Introduction to COH and GISMOL

Constrained Object Hierarchies (COH) is a mathematical framework for representing intelligent systems as a 9-tuple:

```
O = (C, A, M, N, E, I, T, G, D)
```

Each component captures a fundamental aspect of intelligence:

Component	Name	Purpose
C	Components	Hierarchical decomposition into sub-objects
A	Attributes	State variables (observable properties)
M	Methods	Behaviors / actions that change state
N	Neural Components	Learnable functions (e.g., neural networks)
E	Embedding	Maps the whole object to a vector space
I	Identity Constraints	Invariants that must hold in every state
T	Trigger Constraints	Event-condition-action rules for reactive behavior
G	Goal Constraints	Objective functions (rewards / costs)
D	Daemons	Continuous background monitoring processes

GISMOL (General Intelligent Systems Modelling Language) is a Python toolkit that brings COH to life. It provides:

- A clean, object-oriented API for constructing COH objects.
- A simulation engine with discrete-time and event-driven modes.
- Integration with PyTorch for learning.
- Categorical composition operators (product, coproduct, exponential).
- Tools for serialization and visualization.

This tutorial will teach you how to use GISMOL to model, simulate, and train intelligent systems of your own.

---

2. Installation and First Steps

For the latest development version, clone the repository and install with pip:

```
git clone https://github.com/hongxueharriswang/gismol_toolkit.git
cd gismol_toolkit
pip install .
```

Verify the installation by importing the package:

```
import gismol
print(gismol.__version__)   # should print 0.1.0 or later
```

---

3. Your First COH System: A Moving Point

Let's create a very simple system: a point that can move right. We'll define a COH object with a **position** attribute and a **move** method. Then we'll simulate it for a few steps.

Create a file `moving_point.py`:

```
from gismol import COH, Simulator

# Create a COH object with initial position 0
point = COH(name="Point", attributes={"x": 0})

# Define a method that moves the point right by 1
def move_right(state):
    state["x"] += 1
    return state, 0   # reward is 0 (no learning yet)

point.methods["move"] = move_right

# Create a simulator that runs for 5 steps
sim = Simulator(point, dt=1.0, max_steps=5)
sim.run()   # uses a random policy (but there's only one action)

print("Final position:", point.attributes["x"])
```

Run it:

```
python moving_point.py
```

Output:

```
Final position: 5
```

What happened?

- The simulator called the `move` method at each step, incrementing `x` by 1 each time.
- The simulation stopped after 5 steps.
- The final position is 5, as expected.

This demonstrates the core workflow:

1. Create a COH object with attributes and methods.
2. Run a simulation with a policy (here the default random policy chose the only available action).

---

4. The COH 9-tuple in GISMOL – A Detailed Walkthrough

Before building more complex systems, let's explore each component in detail.

Components (C)

Components represent the hierarchical decomposition. A COH object can have **children**, which are themselves COH objects. The parent-child relationships must form a **directed acyclic graph (DAG)**.

```
body = COH(name="Body")
sensor = COH(name="Sensor")
robot = COH(name="Robot", children=[body, sensor])
```

Children are stored in the `children` list. When you add a child using `add_child()`, the child's `parent` attribute is set automatically.

```
robot.add_child(another_part)
```

The toolkit validates the DAG; if you create a cycle, it raises a `ValueError`.

Attributes (A)

Attributes hold the state of an object. They are stored in a dictionary `attributes`. Values can be any JSON-serializable type (numbers, strings, lists, dicts).

```
point.attributes["x"] = 10
point.attributes["color"] = "red"
```

Methods (M)

Methods define the behaviors. A method is a callable that receives the current `attributes` dictionary (and optional arguments) and returns a tuple `(new_attributes, reward)`. The reward is a scalar used for learning (e.g., step cost).

```
def move_up(state):
    x, y = state["position"]
    state["position"] = (x, y+1)
    return state, -1   # step cost

point.methods["up"] = move_up
```

Methods can also call methods on children or access neural components.

Neural Components (N)

Neural components are learnable functions wrapped in `NeuralModule`. They integrate with PyTorch.

```
import torch.nn as nn

class PolicyNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(4, 2)

    def forward(self, x):
        return self.fc(x)

policy = NeuralModule(PolicyNet(), optimizer_class=torch.optim.Adam, lr=1e-
3)
agent.neural["policy"] = policy
```

`NeuralModule` provides `forward()`, `step(loss)`, `train()`, `eval()`, `save()`, and `load()` methods.

Embedding (E)

The embedding function maps a COH object (including its children) to a vector – typically used as input to neural networks.

```
import numpy as np

def my_embedding(coh):
    # Flatten attributes into a vector
    pos = coh.attributes["position"]
    energy = coh.attributes["energy"]
    return np.array([pos[0], pos[1], energy], dtype=np.float32)

agent.embedding = my_embedding
```

If you don't provide an embedding, GISMOL supplies a simple default via `utils.default_embedding` (concatenates all numerical attribute values recursively).

Identity Constraints (I)

Identity constraints are predicates that must hold in **every** state. They are checked after each method execution.

```
def within_bounds(coh):
    x, y = coh.attributes["position"]
    return 0 <= x < 10 and 0 <= y < 10

agent.identity_constraints.append(within_bounds)
```

If a constraint fails, a `ConstraintViolation` exception is raised. You can catch it or let the simulator stop.

Trigger Constraints (T)

Trigger constraints are **event-condition-action** rules. They are evaluated when a matching event occurs.

```
from gismol import Trigger

def low_battery_condition(coh):
    return coh.attributes["energy"] < 20

def recharge_action(coh):
    coh.attributes["energy"] = 100

trigger = Trigger("after_step", low_battery_condition, recharge_action)
agent.trigger_constraints.append(trigger)
```

Events are published by the simulator (e.g., `"step"`, `"after_step"`) or by you via `sim.publish()`.

Goal Constraints (G)

Goal constraints define the objectives. They return a scalar reward (higher is better) for a given state.

```
def goal_reward(coh):
    # reward = negative Manhattan distance to origin
    x, y = coh.attributes["position"]
    return - (abs(x) + abs(y))

agent.goal_constraints.append(goal_reward)
```

Multiple goals can be combined (summed) with weights. The total goal value is obtained by `coh.compute_goal()`.

Daemons (D)

Daemons are background processes that run periodically. They can monitor state, log data, or intervene. Subclass `Daemon` and implement `run()`.

```
from gismol import Daemon

class Logger(Daemon):
    def run(self, coh, dt):
        print(f"Time {coh.parent.time}: pos={coh.attributes['position']}")

agent.daemons.append(Logger(interval=2.0))   # run every 2 time units
```

Daemons are executed by the simulator at the end of each time step, if their interval has elapsed.

---

5. Building a Hierarchy: The Smart Home Example

Let's build a more interesting system: a **smart home** with a thermostat and lights. This will illustrate hierarchical composition.

We'll create two child objects: `Thermostat` and `Lights`, and a root `Home` that contains them.

```
from gismol import COH

# Create components
thermostat = COH(name="Thermostat")
lights = COH(name="Lights")
home = COH(name="Home", children=[thermostat, lights])
```

Now add attributes:

```
thermostat.attributes["temperature"] = 20.0
thermostat.attributes["target"] = 22.0
thermostat.attributes["heater_on"] = False

lights.attributes["bedroom"] = False
lights.attributes["livingroom"] = False

# Add a global "hour" attribute to the home (simulate time of day)
home.attributes["hour"] = 8
```

Define methods:

```
def update_thermostat(state):
    # Simple hysteresis
    if state["temperature"] < state["target"] - 1:
        state["heater_on"] = True
```

```
    elif state["temperature"] > state["target"] + 1:
        state["heater_on"] = False

    if state["heater_on"]:
        state["temperature"] += 0.1
    else:
        state["temperature"] -= 0.05
    return state, -1 if state["heater_on"] else 0

thermostat.methods["update"] = update_thermostat

def toggle_light(state, room):
    state[room] = not state[room]
    return state, -1   # small cost for switching

lights.methods["toggle"] = toggle_light
```

Now we need a way for the home to act. We'll add a root method that calls the child methods according to some policy (simplified here).

```
def home_step(state):
    # Update thermostat
    thermostat.apply_method("update")
    # Maybe toggle a light randomly
    import random
    if random.random() < 0.1:
        room = random.choice(["bedroom", "livingroom"])
        lights.apply_method("toggle", room=room)
    # Advance hour (simulate time passing)
    state["hour"] = (state["hour"] + 1) % 24
    return state, home.compute_goal()   # total goal reward

home.methods["step"] = home_step
```

Now we have a working hierarchical system. We can simulate it with a policy that simply calls `"step"` each iteration.

```
from gismol import Simulator

sim = Simulator(home, dt=1.0, max_steps=50)
sim.run(policy=lambda coh: "step")   # always call the root "step" method
```

This will run the home for 50 simulated hours.

---

6. Adding Constraints to Enforce Behavior

Let's add constraints to make the system smarter and safer.

Identity Constraint: Safe Temperature

We want to ensure the temperature never exceeds 30°C (safety). Add this to the thermostat:

```
def safe_temp(coh):
    return coh.attributes["temperature"] <= 30.0

thermostat.identity_constraints.append(safe_temp)
```

Now if a method ever pushes the temperature above 30, a `ConstraintViolation` will be raised.

Trigger Constraint: Automatic Lights Off at Night

We want lights to turn off automatically after 10 PM. This can be implemented as a trigger on the `lights` object, reacting to the `"step"` event.

```
from gismol import Trigger

def is_late(coh):
    # The home's hour is accessible via parent
    hour = coh.parent.attributes["hour"]
    return hour >= 22

def turn_off_all(coh):
    for room in coh.attributes:
        if isinstance(coh.attributes[room], bool):
            coh.attributes[room] = False

lights.trigger_constraints.append(
    Trigger("step", is_late, turn_off_all)
)
```

Now every time a `"step"` event occurs (at the beginning of each simulation step), if the hour is late, all lights are turned off.

Goal Constraint: Minimize Energy

We want to minimize total energy consumption (heater + lights on). Add this to the home:

```
def energy_goal(coh):
    cost = 0
    # Heater energy
    if coh.children[0].attributes["heater_on"]:
        cost += 1
    # Lights energy: count how many lights are on
    for v in coh.children[1].attributes.values():
        if isinstance(v, bool) and v:
            cost += 1
```

```
    return -cost   # negative because we minimize

home.goal_constraints.append(energy_goal)
```

Now `home.compute_goal()` will return a negative number representing the energy cost.

---

7. Learning with Neural Components

Suppose we want the thermostat to learn the best target temperature to balance comfort and energy. We'll add a neural policy to the thermostat that suggests a new target based on current temperature and hour.

First, define a PyTorch model:

```
import torch.nn as nn
import torch

class SetpointNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(2, 1)   # input: (temp, hour) -> target
adjustment

    def forward(self, x):
        return self.fc(x)   # raw adjustment (to be added to current
target)
```

Wrap it in a `NeuralModule` and attach to the thermostat:

```
from gismol import NeuralModule

setpoint_nn = NeuralModule(SetpointNet(), optimizer_class=torch.optim.Adam,
lr=0.01)
thermostat.neural["setpoint_policy"] = setpoint_nn
```

We need an embedding for the thermostat to provide input to the network. Let's define one:

```
def thermostat_embedding(coh):
    temp = coh.attributes["temperature"]
    hour = coh.parent.attributes["hour"]   # access home's hour
    return np.array([temp, hour], dtype=np.float32)

thermostat.embedding = thermostat_embedding
```

Now modify the thermostat's `update` method to use the neural policy to adjust the target:

```
def update_thermostat(state):
    # Get embedding and forward through network
    emb = thermostat.embedding(thermostat)   # need to pass the object, not
state
    # Actually, inside a method we have only state, not the object.
    # Better: we can access the neural component directly.
    # We'll redesign: the method will use the neural component via the
object.
    # But methods only receive state. We need a way to access the parent
object.
    # One solution: pass the object as an argument, or use a closure.
    # Let's store a reference to the thermostat object inside the method.
    pass
```

To avoid complexity, we can restructure: instead of a method that uses the neural net internally, we can have a separate method that applies the neural net and updates the target. Then the main `update` method uses the current target. Let's do that:

```
def adjust_target(state):
    # Use neural net to propose a new target
    emb = thermostat.embedding(thermostat)   # thermostat is in the closure
    tensor = torch.tensor(emb).float().unsqueeze(0)
    with torch.no_grad():
        delta = thermostat.neural["setpoint_policy"].forward(tensor).item()
    state["target"] += delta
    # Clamp target to reasonable range
    state["target"] = max(18, min(26, state["target"]))
    return state, 0

thermostat.methods["adjust_target"] = adjust_target
```

Then the home's `step` method could call `adjust_target` occasionally.

For learning, we'll use the `ConstrainedRL` learner provided by GISMOL. It expects:

- The root COH object (home) to have an embedding.
- A policy network that outputs logits for each action (number of actions = number of root methods).

But here our root has only one method (`step`). To learn, we might instead treat the thermostat's `adjust_target` as the action. For simplicity, we'll skip detailed RL in this section and refer to the later case study.

---

8. Monitoring with Daemons

Let's add a daemon to log energy usage every 5 steps. Daemons are subclasses of `Daemon` that implement `run()`.

```
from gismol import Daemon

class EnergyLogger(Daemon):
    def run(self, coh, dt):
        # coh is the object the daemon belongs to (here, home)
        heater = coh.children[0].attributes["heater_on"]
        lights_on = sum(1 for v in coh.children[1].attributes.values() if
isinstance(v, bool) and v)
        print(f"Step {coh.parent.step_count}: heater={heater}, lights_on=
{lights_on}, energy={heater + lights_on}")

home.daemons.append(EnergyLogger(interval=5.0))
```

The daemon will run every 5 simulation steps. Note: `coh.parent` in a root object's daemon is the simulator, so we can access `step_count` and `time`.

---

9. Simulation in Depth

The Simulator Class

The `Simulator` is the engine that drives your COH system forward. Its constructor:

```
Simulator(root: COH, dt: float = 1.0, max_steps: Optional[int] = None,
real_time: bool = False)
```

- `root`: the top-level COH object.
- `dt`: time increment per step (simulation time).
- `max_steps`: if set, simulation stops after this many steps.
- `real_time`: if `True`, the simulator sleeps for `dt` seconds between steps (for real-time visualization).

Running the simulation:

```
sim.run(policy=None)   # policy is a callable that returns an action name
```

If no policy is given, a default random policy chooses uniformly from the root's methods.

Policies and Action Selection

A policy is a function that takes the root COH object and returns the name of a method to execute. For example, a policy that always calls `"step"`:

```
def always_step(coh):
    return "step"
```

For systems with multiple root methods, you might implement a more sophisticated policy based on state.

Event Handling

The simulator has an `event_bus` (instance of `EventBus`). You can publish custom events and subscribe to them.

```
def my_handler(event):
    print(f"Received {event.name} with data {event.data}")

sim.event_bus.subscribe("my_event", my_handler)
sim.publish("my_event", data=42)
```

Trigger constraints are automatically subscribed to events matching their `event` pattern.

Built-in events published by the simulator:

- `"step"` – at the beginning of each step.
- `"after_step"` – after the action is applied.
- `"constraint_violated"` – when an identity constraint fails.

Real-Time Simulation

Set `real_time=True` to run in approximate real time. The simulator will call `time.sleep(dt)` after each step. This is useful for visualizations or interactive demos.

---

10. Advanced Composition: Categorical Operations

GISMOL provides categorical operations to combine COH objects in principled ways.

Product (Parallel Composition)

The **product** of two or more COH objects creates a new system where they run in parallel. All components are merged, with attributes and methods prefixed by the object's name to avoid collisions. Identity constraints are combined with logical AND; goals are summed.

```
from gismol.category import product

combined = product(home, another_system)
```

The resulting object has:

- Children: union of all children.
- Attributes: `{ "home.temperature": ..., "another_system.foo": ... }`
- Methods: similarly prefixed.
- Identity constraints: all must hold.
- Goals: sum of goals from each operand.

Coproduct and Exponential (Experimental)

These are placeholders for future releases. Coproduct will represent alternative composition (choice between systems), and exponential will represent function spaces.

---

11. Saving and Loading Your Models

GISMOL provides basic JSON serialization for the hierarchy and attributes. Methods, neural components, and constraints are **not** automatically saved because they are not JSON-serializable.

Saving:

```
from gismol.utils import to_json

to_json(home, "home.json")
```

Loading:

```
from gismol.utils import from_json

loaded_home = from_json("home.json")
# Now you must re-attach methods, constraints, neural components, etc.
```

For full persistence (including neural network weights), you can use Python's `pickle`, but be aware of security implications when loading untrusted data.

```
import pickle
with open("home.pkl", "wb") as f:
    pickle.dump(home, f)
```

---

12. Visualizing the Hierarchy

If you have `matplotlib` installed, you can draw the component DAG:

```
from gismol.visualization import draw_hierarchy

draw_hierarchy(home)                    # interactive display
draw_hierarchy(home, "home_graph.png")   # save to file
```

The graph shows parent-child relationships. It can help you verify that your hierarchy is acyclic and well-structured.

---

13. Debugging and Testing Tips

- **Print statements** in methods, constraints, and daemons are your friends. Use them to trace execution.
- **Test identity constraints separately** by calling `coh.check_identity()` after manual state changes.
- **Use the event bus** to log when triggers fire.
- **Handle `ConstraintViolation`** in a daemon to recover gracefully instead of crashing.
- **Validate the DAG** with `utils.is_dag(coh)` after building the hierarchy.
- **Keep methods pure** (no side effects) to make debugging easier.

---

14. Extending GISMOL

You can extend GISMOL in several ways:

- **Custom constraints** – Subclass `Constraint` and implement `check()`.
- **Custom daemons** – Subclass `Daemon` and override `run()`.
- **New composition operators** – Add functions to `category.py` that return a new COH object.
- **Integration with other ML frameworks** – Create a wrapper similar to `NeuralModule` for JAX, TensorFlow, etc.
- **Alternative simulation engines** – Subclass `Simulator` and override the main loop.

Example of a custom constraint:

```
from gismol.constraints import Constraint

class AlwaysPositive(Constraint):
    def check(self, coh):
        return all(v >= 0 for v in coh.attributes.values())
```

Then add it:

```
home.identity_constraints.append(AlwaysPositive().check)
```

---

15. Complete Case Study: Gridworld Navigator

Let's build the gridworld agent described in the COH paper (Section 8.9). It's a classic example: an agent navigating a grid with obstacles, trying to reach a goal while avoiding obstacles.

We'll implement all nine components and then run a simulation with a random policy and later with a learned policy.

Step 1: Define the hierarchy

We'll have:

- `World` (root) containing:
  - `Grid` (holds dimensions)
  - `Agent` (position)
  - `Obstacles` (set of blocked cells)
  - `Goal` (target position)

```
from gismol import COH

grid = COH(name="Grid", attributes={"width": 5, "height": 5})
agent = COH(name="Agent", attributes={"x": 0, "y": 0})
obstacles = COH(name="Obstacles", attributes={"cells": [(2,2), (3,3)]})
goal = COH(name="Goal", attributes={"pos": (4,4)})

world = COH(name="World", children=[grid, agent, obstacles, goal])
```

Step 2: Define methods (actions)

The agent can move up, down, left, right, or wait. Each method checks for obstacles and updates position.

```
def move(dx, dy):
    def _move(state):
        x, y = agent.attributes["x"], agent.attributes["y"]
        new_x, new_y = x+dx, y+dy
        # Check bounds
        if 0 <= new_x < grid.attributes["width"] and 0 <= new_y <
grid.attributes["height"]:
            # Check obstacle
            if (new_x, new_y) not in obstacles.attributes["cells"]:
                agent.attributes["x"], agent.attributes["y"] = new_x, new_y
        return agent.attributes, -1   # step cost
    return _move

agent.methods["up"] = move(0, 1)
agent.methods["down"] = move(0, -1)
agent.methods["left"] = move(-1, 0)
agent.methods["right"] = move(1, 0)
agent.methods["wait"] = lambda s: (s, -1)
```

We also need a root method to delegate to the agent. Let's add a method to the world that forwards the action to the agent.

```
def world_act(state, action):
    # action is one of "up", "down", "left", "right", "wait"
    agent.apply_method(action)
    return state, world.compute_goal()   # total goal reward

world.methods["act"] = world_act
```

Step 3: Identity constraints

Ensure the agent never occupies an obstacle cell.

```
def not_on_obstacle(coh):
    # coh is the agent
    pos = (coh.attributes["x"], coh.attributes["y"])
    return pos not in obstacles.attributes["cells"]

agent.identity_constraints.append(not_on_obstacle)
```

Also ensure the agent stays within grid bounds (though the move methods already enforce it, it's good to have an invariant).

```
def inside_bounds(coh):
    x, y = coh.attributes["x"], coh.attributes["y"]
    return 0 <= x < grid.attributes["width"] and 0 <= y <
grid.attributes["height"]

agent.identity_constraints.append(inside_bounds)
```

Step 4: Trigger constraints

For demonstration, let's add a trigger that logs when the agent is near the goal.

```
from gismol import Trigger

def near_goal(coh):
    # coh is the agent
    x, y = coh.attributes["x"], coh.attributes["y"]
    gx, gy = goal.attributes["pos"]
    return abs(x-gx) + abs(y-gy) <= 1

def log_near(coh):
    print(f"Agent is near the goal at {coh.attributes['x']},
{coh.attributes['y']}")

agent.trigger_constraints.append(Trigger("after_step", near_goal,
log_near))
```

Step 5: Goal constraints

We want the agent to reach the goal (positive reward) and avoid steps (negative reward). So goal = -distance_to_goal.

```
def goal_reward(coh):
    # coh is the agent
    x, y = coh.attributes["x"], coh.attributes["y"]
    gx, gy = goal.attributes["pos"]
    return - (abs(x-gx) + abs(y-gy))

agent.goal_constraints.append(goal_reward)
```

Step 6: Neural components and embedding

We'll add a simple Q-network to learn the optimal policy. First, an embedding for the agent that includes its position and goal.

```
import numpy as np

def agent_embedding(coh):
    x, y = coh.attributes["x"], coh.attributes["y"]
    gx, gy = goal.attributes["pos"]
    return np.array([x, y, gx, gy], dtype=np.float32)

agent.embedding = agent_embedding
```

Now a neural module:

```
import torch.nn as nn
from gismol import NeuralModule

class QNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(4, 5)   # 4 inputs, 5 actions

    def forward(self, x):
        return self.fc(x)

q_net = NeuralModule(QNet(), optimizer_class=torch.optim.Adam, lr=1e-2)
agent.neural["q_net"] = q_net
```

Step 7: Daemons

Add a simple logger that prints the agent's position every few steps.

```
from gismol import Daemon

class PositionLogger(Daemon):
    def run(self, coh, dt):
```

```
        print(f"Step {coh.parent.step_count}: Agent at
({coh.attributes['x']},{coh.attributes['y']})")

agent.daemons.append(PositionLogger(interval=2.0))
```

Step 8: Simulation with random policy

```
from gismol import Simulator

sim = Simulator(world, dt=1.0, max_steps=50)

def random_policy(coh):
    import random
    return "act"   # but we need to pass the action as an argument to
world_act
```

Wait – our root method `act` expects an action argument. The simulator, when calling a method, does not pass arguments; it only calls the method with the state dict. So we need to adapt: either we define separate root methods for each action (e.g., `world_up`, `world_down`), or we use a policy that calls `apply_method` on the agent directly. Let's do the latter: we'll use a policy that selects an agent action and applies it via the agent. The root doesn't need a method at all.

```
def policy(coh):
    # coh is the world
    import random
    action = random.choice(["up", "down", "left", "right", "wait"])
    agent.apply_method(action)
    # No need to return an action name for the root
    return None   # simulator will not call any root method
```

But the simulator expects a policy that returns a method name for the root. If we return `None`, it will do nothing. Alternatively, we can give the root a dummy method that does nothing and always call that. Simpler: let's define a root method that does the delegation.

```
def world_step(state):
    # We'll use a global variable to remember the chosen action? Not good.
    # Better: have the policy set an attribute in the world indicating the
chosen action.
    pass
```

Actually, the simulator's design assumes the root's methods are the actions. So to use the agent's methods directly, we could make the root have methods that mirror the agent's actions. Let's do that:

```
def make_world_action(action_name):
    def _action(state):
        agent.apply_method(action_name)
        return state, world.compute_goal()
    return _action

for act in ["up", "down", "left", "right", "wait"]:
    world.methods[act] = make_world_action(act)
```

Now the root has five methods. A policy can return one of these names.

```
def random_policy(coh):
    import random
    return random.choice(["up", "down", "left", "right", "wait"])
```

Run the simulation:

```
sim = Simulator(world, dt=1.0, max_steps=50)
sim.run(policy=random_policy)
```

Step 9: Learning with Q-learning

We can use the `ConstrainedRL` learner, but it expects a policy network that outputs logits for actions. Our Q-network outputs Q-values; we can use epsilon-greedy for action selection. Let's implement a simple Q-learning loop manually.

```
import torch

epsilon = 0.1
gamma = 0.9

for episode in range(100):
    # Reset agent to start (0,0)
    agent.attributes["x"], agent.attributes["y"] = 0, 0
    total_reward = 0
    step = 0
    while step < 50:
        # Get state embedding
        s = torch.tensor(agent.embedding(agent)).float().unsqueeze(0)
        # Choose action epsilon-greedy
        if np.random.random() < epsilon:
            action = np.random.choice(["up","down","left","right","wait"])
            action_idx = ["up","down","left","right","wait"].index(action)
        else:
            with torch.no_grad():
                q_values = agent.neural["q_net"].forward(s)
```

```
                action_idx = torch.argmax(q_values).item()
                action = ["up","down","left","right","wait"][action_idx]
        # Apply action
        agent.apply_method(action)
        # Observe reward and next state
        reward = agent.compute_goal()   # actually we want step reward;
here goal is negative distance
        # For learning, we need a proper reward: maybe -1 per step, +50
when goal reached.
        # Let's define a simpler reward: -1 per step, +100 when on goal.
        on_goal = (agent.attributes["x"], agent.attributes["y"]) ==
goal.attributes["pos"]
        reward = -1 + (100 if on_goal else 0)
        s_next = torch.tensor(agent.embedding(agent)).float().unsqueeze(0)
        # Compute target
        with torch.no_grad():
            q_next = agent.neural["q_net"].forward(s_next)
            target = reward + gamma * torch.max(q_next)
        # Current Q
        q_current = agent.neural["q_net"].forward(s)[0, action_idx]
        loss = (q_current - target) ** 2
        # Update
        agent.neural["q_net"].step(loss)
        total_reward += reward
        step += 1
        if on_goal:
            break
    print(f"Episode {episode}, total reward: {total_reward}")
```

This is a basic Q-learning implementation. You can refine it with experience replay, target networks, etc.

---

16. Conclusion and Next Steps

Congratulations! You've learned how to use GISMOL to build intelligent systems based on the COH 9-tuple framework. You've seen how to:

- Create COH objects with attributes and methods.
- Compose them hierarchically.
- Enforce behavior with identity, trigger, and goal constraints.
- Integrate neural networks for learning.
- Monitor systems with daemons.
- Simulate and visualize the results.

Where to go from here:

- Explore the `examples/` directory in the GISMOL repository for more complex systems (quantum control, biology, smart city, finance, cultural heritage).
- Read the original COH paper for deeper theoretical insights.
- Contribute to GISMOL by adding new features or examples.
- Apply GISMOL to your own research or projects – it's designed to be domain-agnostic.

We hope this tutorial has empowered you to model and understand intelligent systems in a principled, modular way. Happy modelling!

For questions, issues, or contributions, visit https://github.com/hongxueharriswang/gismol_toolkit.